TCG Digital — SRE Guild Lead

120 Strong Second-Round Interview Questions & Precise Spoken Answers

Updated edition: The previous PDF contained 98 entries because the final two answers were omitted during PDF generation. This edition restores the missing 2 and adds 20 additional production-style scenario questions.

Interview focus: Guild leadership and influence without authority, SLO/error-budget governance, incident command, observability, Kubernetes, CI/CD, IaC, automation, DR/chaos, architecture, mentoring, capacity/cost and measurable outcomes.

Memorization method: Memorize the answer first, then the Remember line. For scenario questions, speak the sequence and then add one real production example from your experience.

1. How would you establish an SRE Guild across 12 pods?

Answer: I would assess each pod's reliability maturity, incidents, SLOs, observability, on-call and automation. Then I'd define minimum standards, pilot them with one pod, measure results, refine, and scale across the pods.

Remember: Assess → Standards → Pilot → Measure → Scale

2. How do you influence teams when you have no direct authority?

Answer: I use technical credibility, data and collaboration. I understand the pod's pain points, propose practical standards, pilot them, show measurable results, and use those results to gain adoption rather than imposing standards.

Remember: Data → Credibility → Pilot → Results → Adoption

3. What would your SRE Guild Charter contain?

Answer: It would define purpose, scope, non-negotiable reliability standards, engagement model, governance, responsibilities, exceptions and success metrics. The charter should enable pods without taking away their delivery ownership.

Remember: Purpose → Scope → Standards → Governance → Metrics

4. How would you standardize SRE across different technologies?

Answer: I would standardize reliability outcomes, not force one tool. Every production service should have meaningful SLOs, actionable alerts, dashboards, runbooks, on-call ownership and incident practices, while teams can choose suitable tools.

Remember: Outcomes, not tools

5. How would you assess the maturity of all 12 pods?

Answer: I would assess SLOs, incident management, observability, alert quality, automation, on-call, documentation and deployment reliability. I'd create a maturity scorecard, identify gaps, prioritize high-risk areas and track improvement.

Remember: Assess → Score → Prioritize → Improve

6. What would you do in your first 30–60–90 days?

Answer: First 30 days: understand teams, applications and pain points. Next 30: define the charter, standards and pilot. By 90 days: demonstrate measurable results and establish the roadmap for wider adoption.

Remember: Understand → Standardize → Prove → Scale

7. What if a pod lead says they don't have time for observability standards?

Answer: I would first understand the constraint and connect the standard to their pain points, such as fewer pages and faster troubleshooting. I'd reduce adoption effort through templates and agree on a practical timeline.

Remember: Understand → Value → Simplify → Timeline

8. Two pods use different monitoring tools. What would you do?

Answer: I would not force immediate consolidation. I'd define common observability requirements and reliability outcomes. If standardizing a tool provides clear value, I'd prove it through a pilot before recommending adoption.

Remember: Outcome first, tool second

9. How would you avoid the Guild becoming a bottleneck?

Answer: I'd create reusable golden paths, templates, Terraform modules, runbooks and self-service patterns. The Guild defines standards and enables teams, while pods retain operational ownership.

Remember: Enable, don't control

10. How would you handle a pod that refuses Guild standards?

Answer: I'd understand their constraints, use data to explain the reliability risk, provide a migration path and agree on a timeline. If a serious risk remains, I'd use the agreed governance and escalation path.

Remember: Understand → Evidence → Support → Timeline → Escalate

11. What if a pod says, 'We already do SRE'?

Answer: I would assess their practices against the maturity framework, recognize what they do well, identify gaps and agree on improvements. The objective is to increase maturity, not impose a process.

Remember: Assess, don't impose

12. How would you get engineers to adopt SRE standards?

Answer: I'd involve engineers in defining practical standards, provide reusable templates, pilot them with a real pod and show measurable benefits. Adoption improves when engineers see the standards solving production problems.

Remember: Involve → Simplify → Pilot → Prove

13. How do you handle disagreement between two senior engineers?

Answer: I'd bring the discussion back to requirements, reliability risk and measurable data. I'd hear both sides, evaluate trade-offs objectively and make a decision based on system and business needs.

Remember: Requirements → Data → Trade-offs → Decision

14. How do you make engineers accountable for reliability?

Answer: I make service ownership explicit. Each pod owns its SLOs, alerts, runbooks and incident actions, while the Guild provides standards, coaching, tooling patterns and governance.

Remember: Pod owns → Guild enables

15. What is the biggest mistake an SRE Guild Lead can make?

Answer: Treating the Guild as a centralized operations team and imposing standards without understanding pod realities. The Guild should enable teams, create consistency and influence through measurable outcomes.

Remember: Don't become the bottleneck

16. Why should the Guild exist if DevOps and Infrastructure already exist?

Answer: DevOps and Infrastructure provide platform and operational capabilities, while the SRE Guild creates a consistent reliability practice across delivery pods through SLOs, incidents, observability and automation.

Remember: Platform → Reliability practice

17. What is SLI, SLO and SLA?

Answer: SLI is the actual reliability measurement, SLO is the internal target, and SLA is the external commitment. For example, availability is the SLI, 99.9% is the SLO, and a contractual availability commitment is the SLA.

Remember: Measure → Target → Contract

18. What makes a good SLI?

Answer: A good SLI represents customer experience and is consistently measurable. For an API, successful request rate and latency are common examples.

Remember: Customer experience + measurable

19. Availability or latency—which is more important?

Answer: Both can matter. Availability tells us whether the service works, while latency tells us whether it works within an acceptable response time. The choice depends on the critical user journey.

Remember: Works? → Availability | Fast enough? → Latency

20. What is an error budget?

Answer: Error budget is the unreliability allowed by the SLO. For a 99.9% availability SLO, the remaining 0.1% is the error budget used to balance reliability and delivery.

Remember: 100% – SLO = Error Budget

21. What is burn rate?

Answer: Burn rate shows how quickly the error budget is being consumed. A high burn rate means reliability is deteriorating faster than expected and needs investigation or release control.

Remember: How fast are we spending reliability?

22. What if an SLO is too strict?

Answer: I would validate it against historical performance, customer expectations and business requirements. If it is unrealistic, I'd adjust it through the agreed governance process using data.

Remember: Data → Business → Adjust

23. What if an SLO is too relaxed?

Answer: I'd verify whether it represents meaningful customer experience. If not, I'd use customer impact and historical data to propose a stronger target.

Remember: Meaningful customer experience

24. What if a shared dependency causes multiple SLO breaches?

Answer: I'd identify it as a common failure domain, establish ownership and reliability expectations, and make sure dependent services account for that dependency in their reliability planning.

Remember: Common dependency → Common ownership

25. What if a service consumes 80–90% of its error budget?

Answer: I'd identify the main causes and burn rate. If reliability is deteriorating, I'd prioritize reliability work and potentially slow risky releases until the service stabilizes.

Remember: Find cause → Burn rate → Reliability work

26. What if the error budget is exhausted?

Answer: I'd pause non-critical or risky releases and prioritize reliability improvements. After fixing major causes and restoring acceptable reliability, normal feature delivery can resume.

Remember: Pause → Fix → Stabilize → Resume

27. How do you use SLOs to make release decisions?

Answer: If reliability is within the SLO and error budget is healthy, delivery can continue. If the budget is being consumed rapidly, reliability work and safer release practices take priority.

Remember: SLO + Error Budget = objective decision

28. What if product leadership disagrees with an error-budget policy?

Answer: I'd explain the policy using customer impact and reliability data, discuss business trade-offs, and agree on an explicit decision with leadership rather than making reliability an opinion-based debate.

Remember: Data → Trade-off → Explicit decision

29. How would you roll up SLO health across multiple pods?

Answer: I'd standardize SLO definitions and reporting, then aggregate attainment, error-budget consumption and burn-rate trends by service, pod and business criticality.

Remember: Standardize → Aggregate → Report

30. How would you reduce alert fatigue?

Answer: I'd analyze volume, duplicates, false positives and non-actionable alerts. Then I'd classify paging versus ticket alerts, tune thresholds, remove duplicates, add runbooks and measure the reduction.

Remember: Analyze → Classify → Tune → Runbook → Measure

31. How do you design an actionable alert?

Answer: It should represent a meaningful reliability risk, have an appropriate threshold and severity, identify ownership and provide a clear response path. Critical alerts should have runbooks.

Remember: Meaningful → Threshold → Owner → Action → Runbook

32. What should page an engineer versus create a ticket?

Answer: Pages are for urgent conditions requiring immediate human action, usually customer impact or imminent impact. Lower-priority issues should go to tickets or dashboards.

Remember: Urgent = Page | Non-urgent = Ticket

33. How would you standardize observability?

Answer: I'd define minimum coverage for metrics, logs, traces, dashboards, golden signals and alerting. Critical services need actionable alerts, clear ownership and runbooks.

Remember: Metrics → Logs → Traces → Alerts → Dashboard → Runbook

34. What are the golden signals?

Answer: The four golden signals are latency, traffic, errors and saturation. They provide a practical view of user experience, workload and resource health.

Remember: Latency → Traffic → Errors → Saturation

35. Monitoring versus observability?

Answer: Monitoring tells us whether known conditions are healthy. Observability helps us understand why the system behaves that way using metrics, logs and traces.

Remember: Monitoring = What | Observability = Why

36. Dashboard is green but customers report checkout failures. What do you do?

Answer: I'd validate the customer impact and trace the complete transaction across application, dependencies and external services using logs, metrics and traces. It may indicate an observability gap.

Remember: User impact beats green dashboard

37. How would you investigate high latency?

Answer: I'd determine whether it is global or isolated, then check application metrics, infrastructure saturation, database performance, dependency latency, network metrics and traces to locate where latency is introduced.

Remember: Scope → App → Infra → DB → Dependency → Network → Trace

38. How do you prove alert-fatigue reduction?

Answer: I'd baseline alert volume, duplicates, false positives, pages per engineer and alert-driven MTTR. After tuning, I'd compare the metrics and verify critical incidents are still detected.

Remember: Baseline → Tune → Compare → Validate

39. What are your first five minutes of a Sev-1?

Answer: Confirm impact and severity, establish incident ownership, open communication, identify affected services and start diagnosis using metrics and logs. The immediate priority is restoring customer service.

Remember: Impact → Owner → Communicate → Diagnose → Restore

40. What is the role of an Incident Commander?

Answer: The Incident Commander coordinates the response rather than doing all technical work. They assign owners, control priorities, maintain communication and drive the team toward mitigation and recovery.

Remember: Coordinate, don't troubleshoot everything

41. Rollback or fix forward?

Answer: If a recent deployment clearly caused the incident and rollback is safe, I prefer rollback to restore the known-good state quickly. I fix forward when rollback is unsafe or a safe fix is faster.

Remember: Fastest safe recovery

42. How do you decide incident severity?

Answer: I consider customer impact, affected users, business criticality, geographic scope, duration and data or security impact. Severity should be based on impact, not just the failed component.

Remember: Impact determines severity

43. What if the same incident keeps recurring?

Answer: I'd review previous RCAs and identify why corrective actions failed. Then I'd prioritize permanent fixes, automation or architectural changes and track them to closure.

Remember: Repeated incident → Permanent fix

44. What makes a good postmortem?

Answer: It explains the timeline, customer impact, detection and response, root and contributing causes, and corrective actions with owners and deadlines. It should be blameless and prevention-focused.

Remember: Timeline → Impact → Cause → Actions → Prevention

45. What if an engineer caused the incident?

Answer: I'd keep the postmortem blameless and focus on system conditions that allowed the mistake to cause impact. Then I'd improve testing, safeguards, approvals or automation.

Remember: Fix the system, not the person

46. How would you handle increasing MTTR?

Answer: I'd segment incidents by service, severity, failure type and response stage to find where time is lost. Then I'd improve detection, runbooks, ownership, escalation and automation.

Remember: Segment → Find delay → Improve → Automate → Measure

47. How would you design an on-call model?

Answer: I'd define service ownership, primary and secondary on-call, escalation paths, coverage, handovers and runbooks. I'd monitor on-call load and adjust staffing based on incident and page trends.

Remember: Owner → Primary/Secondary → Escalate → Load

48. What if engineers are overloaded by on-call?

Answer: I'd analyze pages, incident volume and after-hours load, remove noisy alerts, automate repetitive work and review staffing or service ownership if the load remains unsustainable.

Remember: Reduce pages → Automate → Staff appropriately

49. How would you conduct a blameless RCA?

Answer: I focus on timeline, technical and organizational contributing factors, detection gaps and why controls failed. Actions should improve the system rather than assign individual blame.

Remember: Timeline → Causes → Controls → Actions

50. Production fails immediately after deployment. What do you do?

Answer: I'd correlate the failure timeline with the deployment, assess impact and check application and infrastructure signals. If the deployment is the likely cause, I'd rollback, recover service, then perform RCA.

Remember: Correlate → Impact → Rollback → Recover → RCA

51. How do you handle a Kubernetes CrashLoopBackOff?

Answer: I'd check pod status, `kubectl describe pod`, `kubectl logs --previous` and events. Then I'd check configuration, dependencies, OOMKilled, resource limits and probes. If release-related, I'd consider rollback.

Remember: get pods → describe → logs --previous → events → resources/probes → rollback

52. Pods are Pending. What do you check?

Answer: I'd check `kubectl describe pod` and events, then node capacity, CPU and memory requests, taints and tolerations, affinity, quotas and autoscaler status.

Remember: describe → events → capacity → requests → taints → autoscaler

53. Pods are OOMKilled. What do you check?

Answer: I'd compare memory usage with requests and limits, inspect restart history and logs, and determine whether the limit is too low or the application has a memory leak.

Remember: Usage → Limit → Logs → Leak

54. HPA is not scaling correctly. What do you check?

Answer: I'd check `kubectl describe hpa`, current versus target metrics, metrics-server or custom metrics, resource requests and available capacity.

Remember: HPA → Metrics → Requests → Capacity

55. Kubernetes service returns 502/504. What do you check?

Answer: I'd trace ingress or load balancer to service and pods. I'd check endpoints, readiness, application logs, network policies, upstream latency and timeout settings.

Remember: Ingress → Service → Endpoints → Pods → App → Network

56. How would you create a Kubernetes golden path?

Answer: I'd provide a reusable Helm base with standard labels, resource requests and limits, health probes, PDB, security settings and observability configuration, while allowing application-specific configuration.

Remember: Helm → Resources → Probes → PDB → Security → Observability

57. How would you safely upgrade Kubernetes?

Answer: I'd validate compatibility, test in lower environments, review workloads and add-ons, then upgrade in stages. I'd monitor each stage and maintain a recovery plan.

Remember: Compatibility → Test → Stage → Monitor → Recover

58. How would you standardize CI/CD across pods?

Answer: I'd define a common pipeline framework for build, testing, security scanning, artifacts, deployment, approvals and rollback. Teams can customize application-specific stages while keeping core controls consistent.

Remember: Common controls + team flexibility

59. What quality gates would you implement?

Answer: I'd use unit tests, code quality and coverage checks such as SonarQube, vulnerability scanning such as Trivy, and deployment validation. Critical security or quality failures should block production.

Remember: Test → Quality → Security → Validation

60. How would you improve deployment reliability?

Answer: I'd use automated testing, quality gates, progressive deployment where appropriate, health checks, deployment monitoring and automated rollback. I'd also track deployment failure and rollback rates.

Remember: Test → Gate → Progressive → Monitor → Rollback

61. How would you migrate a manual deployment to CI/CD?

Answer: I'd document the existing process, identify manual steps, automate incrementally, add testing and security gates, validate in lower environments and then introduce controlled production deployment.

Remember: Document → Automate → Test → Validate → Production

62. How would you use progressive delivery?

Answer: I'd release to a small percentage or limited environment first, monitor SLOs and key health metrics, and automatically or manually stop or roll back if the canary shows unacceptable degradation.

Remember: Small blast radius + abort plan

63. How would you identify toil?

Answer: I look for repetitive, manual, predictable and automatable operational work. I quantify engineer effort and prioritize automation based on frequency, risk and business value.

Remember: Repetitive → Measure → Prioritize → Automate

64. When should you not automate?

Answer: I'd avoid automating tasks that are very rare, unstable, poorly understood or cheaper to perform manually. First understand and standardize the process, then automate when the return justifies it.

Remember: Understand first, automate second

65. How do you measure automation success?

Answer: I'd measure manual hours eliminated, execution frequency, failure rate, MTTR improvement and engineer effort saved. Automation is successful when it reduces toil without adding operational risk.

Remember: Hours saved + Reliability

66. How would you create reusable Terraform standards?

Answer: I'd create versioned reusable modules with secure defaults, naming, tagging, networking and monitoring standards. Teams consume the modules while retaining controlled configuration flexibility.

Remember: Reusable → Secure → Versioned → Controlled

67. How would you prevent accidental Terraform destroy?

Answer: I'd use state controls, code review, CI/CD approvals and protected production workflows. Destructive operations on critical infrastructure should require explicit approval.

Remember: Review → Protect → Approve

68. How would you govern infrastructure changes?

Answer: I'd use IaC, pull-request reviews, automated validation, policy checks and controlled deployment pipelines. Production changes should be auditable, repeatable and protected by approvals.

Remember: Code → Review → Validate → Approve → Audit

69. How would you design DR for a Tier-1 application?

Answer: I'd start with business-defined RTO and RPO, then choose multi-zone or multi-region architecture. I'd cover compute, data, networking and dependencies and regularly test recovery.

Remember: RTO/RPO → Architecture → Dependencies → Test

70. How do you prove DR works?

Answer: I'd conduct controlled failover exercises, measure actual RTO and RPO, validate application functionality and dependencies, document gaps and track remediation to closure.

Remember: Failover → Measure → Validate → Fix

71. What is your approach to chaos engineering?

Answer: I start with a hypothesis and defined steady state. Then I introduce a controlled failure such as pod termination or resource stress, observe the response and verify resilience mechanisms.

Remember: Hypothesis → Failure → Observe → Validate

72. How do you make chaos experiments safe?

Answer: I start in non-production, define blast radius and abort conditions, obtain approvals and monitor continuously. Production experiments should be limited in scope with clear recovery procedures.

Remember: Small blast radius + abort plan

73. Traffic suddenly increases 3x. What do you do?

Answer: I'd assess customer impact and saturation, verify autoscaling and capacity, check database and dependency bottlenecks, scale where needed and monitor latency, errors and saturation.

Remember: Impact → Saturation → Scale → Bottleneck → Monitor

74. CPU is low but latency is high. Why?

Answer: CPU alone does not show overall health. I'd investigate database latency, I/O, network, downstream dependencies, locks, connection pools and application bottlenecks using metrics, logs and traces.

Remember: Low CPU ≠ Healthy

75. How would you balance reliability and cloud cost?

Answer: I'd understand the workload and reliability requirement, then optimize utilization without violating SLOs. Right-sizing, requests and limits, autoscaling and cost attribution can reduce waste.

Remember: SLO first → Optimize cost

76. What NFRs do you consider in architecture review?

Answer: I consider availability, scalability, performance, security, resilience, observability, maintainability, disaster recovery and cost, with measurable requirements wherever possible.

Remember: Availability → Scale → Performance → Security → Resilience → Observability

77. How do you handle dependency failure?

Answer: I'd use appropriate timeouts, bounded retries with backoff, circuit breakers and graceful degradation to prevent one dependency failure from cascading across the application.

Remember: Timeout → Retry → Circuit breaker → Degrade

78. Why can retries be dangerous?

Answer: Uncontrolled retries can multiply traffic during an outage and create a retry storm. I'd use bounded retries, exponential backoff, jitter and circuit breakers where appropriate.

Remember: Retries can amplify failures

79. What governance model would you establish?

Answer: I'd define minimum standards, ownership, exception management, maturity reviews and regular Guild reviews. Governance should provide consistency without taking operational ownership away from pods.

Remember: Standards → Ownership → Exceptions → Review

80. How often would you review Guild health?

Answer: I'd use regular operational reviews and monthly or quarterly maturity reviews. Critical incident and SLO trends should be reviewed more frequently.

Remember: Operational metrics frequent; maturity strategic

81. What would trigger escalation to leadership?

Answer: Repeated critical incidents, sustained SLO breaches, rapid error-budget burn, major customer impact, unresolved high-risk actions or capacity gaps would trigger escalation.

Remember: Risk + Trend + Customer impact

82. What would your Guild dashboard contain?

Answer: I'd show P1/P2 trends, MTTR, MTTD, SLO attainment, error-budget burn, alert noise, deployment failures, automation coverage, recurring incidents and remediation status.

Remember: Incidents → SLO → Alerts → Deployments → Automation → Actions

83. How would you measure Guild success?

Answer: I'd expect fewer repeated incidents, lower MTTR, better SLO attainment, reduced alert noise, increased automation and stronger engineering ownership. The goal is measurable reliability improvement.

Remember: Reliability + Efficiency + Business Impact

84. How would you measure business value?

Answer: I'd connect engineering metrics to outcomes: fewer customer-impacting incidents, lower MTTR, better availability, fewer failed deployments and reduced manual toil.

Remember: Learn → Adjust → Re-pilot

85. What if your first Guild initiative fails?

Answer: I'd treat it as feedback, understand why it failed, collect data, adjust the approach, run another pilot and communicate the lessons transparently.

Remember: Teach → Practice → Ownership → Independence

86. How would you mentor engineers across pods?

Answer: I'd use production incidents, troubleshooting sessions, code reviews and knowledge-sharing. I'd focus on developing independent decision-making and gradually increasing ownership.

Remember: Risk + Load + Criticality → Leadership

87. How would you handle capacity or staffing gaps?

Answer: I'd use incident volume, on-call load, service criticality, SLO risk and roadmap demand to quantify the gap, then present the impact and recommended staffing or prioritization to leadership.

Remember: Competency → Evidence → Consistency

88. How would you contribute to hiring for SRE?

Answer: I'd align hiring with the competency model, assess production troubleshooting, SLOs, observability, automation, Kubernetes and incident leadership, and use consistent technical evaluation across candidates.

Remember: Early design → NFRs → Measure

89. How would you build an SRE competency framework?

Answer: I'd define levels across reliability fundamentals, incident management, observability, automation, cloud, Kubernetes, architecture and leadership, with observable behaviors and ownership expectations at each level.

Remember: Platform + Reliability alignment

90. How would you partner with Architecture?

Answer: I'd engage early in design reviews and validate NFRs such as scalability, fault tolerance, observability and DR. I'd ensure reliability requirements are measurable before production.

Remember: Problem → Criteria → Pilot → Measure → Standardize

91. How would you partner with DevOps and Infra?

Answer: I'd align on infrastructure, CI/CD, platform reliability, capacity and security standards. The goal is a shared platform that makes reliable engineering easier for delivery pods.

Remember: Capabilities → Rationalize → Evidence

92. How would you introduce a new SRE tool?

Answer: I'd first identify a clear problem and success criteria, compare alternatives, run a controlled pilot, measure the outcome and standardize only if it improves reliability or reduces cost.

Remember: Risk → Data → Safe release decision

93. How would you handle tool sprawl?

Answer: I'd define required capabilities and supported patterns rather than immediately banning tools. I'd identify overlap, operational cost and reliability impact, then rationalize tools based on evidence.

Remember: Prioritize → Quick wins → Roadmap

94. How would you handle a delivery deadline that conflicts with reliability work?

Answer: I'd quantify the reliability risk using SLO and error-budget data, discuss the trade-off with delivery leadership and agree on the safest release plan rather than making an unsupported yes/no decision.

Remember: Learn → Improve → Repeat

95. What if leadership wants immediate reliability improvement?

Answer: I'd prioritize the highest-impact reliability problems, deliver a few measurable quick wins and build a longer-term roadmap. I would avoid trying to transform everything simultaneously.

Remember: Short → Actionable → Owned → Tested

96. How would you build a culture of continuous improvement?

Answer: I'd use incident reviews, postmortems, reliability metrics, Guild knowledge sharing and recurring improvement actions. Teams should learn from production data and continuously reduce recurring failure and toil.

Remember: Value → Effort → Automate/Simplify

97. How would you ensure runbooks remain useful?

Answer: I'd keep them short and actionable, assign ownership, review them after incidents and test critical procedures periodically. Runbook quality should be part of service maturity.

Remember: Baseline first → Roadmap

98. What would you do if an SRE standard increases delivery effort?

Answer: I'd determine whether the standard provides enough reliability value, simplify or automate it where possible, and adjust the standard if the cost clearly outweighs the benefit.

Remember: Hands-on → Standards → Mentoring → Scale

99. What would your first priority be if joining this greenfield Guild?

Answer: I'd establish the baseline: services, ownership, incidents, SLO maturity, observability, on-call, automation and major reliability risks. That gives me evidence to build the Guild roadmap.

Remember: Baseline → Risk → Roadmap

100. Why are you suitable for this SRE Guild Lead role?

Answer: I bring strong hands-on SRE and DevOps experience across production incidents, observability, Kubernetes, CI/CD, automation and reliability practices. I can convert that experience into standards, mentoring and measurable practices that scale across pods.

Remember: Hands-on → Standards → Mentoring → Scale

101. P99 latency suddenly increases 5x. Walk me through your response.

Answer: I'd first confirm customer impact and whether the spike correlates with traffic or a recent deployment. Then I'd check errors, saturation, dependencies and traces. If deployment-related, I'd rollback safely; otherwise I'd mitigate by scaling or isolating the bottleneck while continuing RCA.

Remember: Impact → Correlate → Diagnose → Mitigate → RCA

102. A payment provider is down. What do you do?

Answer: I'd confirm the dependency failure and prevent cascading impact using timeouts, bounded retries and circuit breaking. If supported, I'd use a fallback or graceful degradation, communicate customer impact and monitor recovery. I'd avoid retry storms and verify idempotency.

Remember: Detect → Contain → Degrade → Communicate → Recover

103. Your database is the bottleneck during peak traffic. What do you do?

Answer: I'd confirm database saturation using connections, CPU, I/O, locks, query latency and throughput. I'd mitigate with connection control, caching or scaling where safe, then optimize expensive queries and review capacity planning for the next peak.

Remember: Metrics → Mitigate → Optimize → Capacity

104. A new release increases 5xx errors but only in one region. What do you do?

Answer: I'd compare that region with healthy regions and correlate the issue with deployment, configuration, traffic and dependencies. I'd stop the rollout, route traffic away if safe, rollback the affected region and then investigate the regional difference.

Remember: Compare → Stop rollout → Shift traffic → Rollback → RCA

105. Kubernetes nodes show high CPU, but pods are healthy. What do you investigate?

Answer: I'd check node-level CPU consumers, daemonsets, system workloads, requests versus actual usage and autoscaler behaviour. I'd determine whether capacity is genuinely insufficient or whether a workload is misconfigured before simply adding nodes.

Remember: Node → Workloads → Requests → Autoscaling → Root cause

106. Pods are healthy but users receive intermittent 503s. What do you check?

Answer: I'd trace the request path through ingress, service and endpoints. I'd check readiness flapping, endpoint changes, load-balancer health, connection limits and application errors, then correlate failures with specific pods or zones.

Remember: Ingress → Service → Endpoints → Readiness → Pods

107. A memory leak causes gradual pod restarts every few hours. How would you detect it?

Answer: I'd trend container memory over time rather than only looking at current usage. I'd correlate memory growth with restarts and traffic, use application metrics or profiling where available, and fix the leak rather than simply increasing limits.

Remember: Trend → Correlate → Profile → Fix

108. A critical alert fires every five minutes but never requires action. What do you do?

Answer: I'd classify it as noisy, identify the underlying condition, tune or remove the alert and replace it with a meaningful SLO or burn-rate signal where appropriate. I'd verify that genuine incidents remain detectable before closing the change.

Remember: Classify → Tune → Replace → Validate

109. A service has no useful SLOs and the pod asks you to define them tomorrow. What do you do?

Answer: I'd start with the most important customer journey, select a small number of user-facing SLIs, use available baseline data and agree on an initial achievable target. I'd explicitly treat it as a baseline and refine it with evidence.

Remember: Journey → SLI → Baseline → Initial SLO → Refine

110. 80% of the error budget is consumed halfway through the window. What is your decision?

Answer: I'd review the burn causes and whether the consumption is from incidents, releases or dependencies. If the policy defines 80% as a trigger, I'd restrict high-risk changes, prioritize reliability work and align with product on the remaining release plan.

Remember: Burn cause → Policy → Risk control → Reliability

111. A Sev-1 has three senior engineers giving conflicting technical directions. What do you do?

Answer: As Incident Commander, I'd assign clear roles, establish one decision path and ask for evidence from each hypothesis. I'd choose the safest mitigation based on customer impact and reversibility, while keeping deeper investigation parallel.

Remember: IC → Roles → Evidence → Safest mitigation

112. The incident is fixed, but stakeholders are unhappy with communication. What do you improve?

Answer: I'd review the incident communication timeline and identify where updates were delayed, unclear or missing ownership. I'd standardize stakeholder updates, escalation paths and communication templates and include those actions in the postmortem.

Remember: Review → Gap → Standardize → Improve

113. A team wants to use a new observability platform. How do you evaluate it?

Answer: I'd define the problem and success criteria first, then compare coverage, integration, operational effort, cost and reliability value. I'd run a limited pilot and adopt it only if measurable benefits justify the change.

Remember: Problem → Criteria → Pilot → Measure → Decide

114. Product wants to release a risky feature while the service is burning error budget. What do you say?

Answer: I'd present the current SLO and burn-rate evidence and explain the reliability risk. I'd propose safer options such as a canary, limited rollout or deferral, and make the final trade-off explicit with product leadership.

Remember: Evidence → Risk → Safer rollout → Decision

115. A Terraform change would recreate a critical production database. What do you do?

Answer: I would stop the change, inspect the plan and identify why Terraform wants to recreate the resource. I'd use protected workflows, review the state and dependencies, and require explicit approval before any destructive production action.

Remember: Stop → Plan → State → Dependencies → Approval

116. A Kubernetes deployment succeeds, but application startup takes too long and readiness fails. What do you do?

Answer: I'd check startup duration, readiness configuration, dependency latency and resource limits. If the application is healthy but slow to initialize, I'd use an appropriate startup probe and tune readiness rather than weakening the checks blindly.

Remember: Startup → Probe → Dependency → Resources → Tune

117. One pod is much slower than its replicas. How do you isolate it?

Answer: I'd compare that pod's latency, CPU, memory, logs, node placement, network path and dependency calls with healthy replicas. If the issue is isolated, I'd remove the unhealthy pod from service while investigating the underlying difference.

Remember: Compare replicas → Isolate → Remove from traffic → Diagnose

118. A third-party API has high latency but no errors. How do you protect your service?

Answer: I'd use strict timeouts, bounded retries with backoff, circuit breaking where appropriate and graceful degradation. I'd monitor dependency latency separately and ensure slow downstream calls cannot exhaust application threads or connection pools.

Remember: Timeout → Bound retries → Circuit breaker → Degrade

119. Leadership asks why MTTR improved but incident count did not. How do you explain it?

Answer: It means response and recovery became faster, but the underlying failure frequency has not improved. I'd use recurring-incident analysis to target prevention while continuing to preserve the response improvements.

Remember: MTTR = response | Count = prevention

120. What would make you change an SRE standard after rollout?

Answer: I'd change it when production data shows the standard is ineffective, too costly, creates unintended risk or no longer fits the architecture. I'd use evidence, feedback and a controlled revision process rather than changing standards informally.

Remember: Evidence → Feedback → Revise → Govern